

Part I — Foundations

Why verification exists, how a verifier thinks, why completeness is formally impossible — and the lifecycle that manages the residual risk instead of pretending it away.

Chapters **1–4** · Running example: **the reference SoC** · Every figure carries its year and its source

Four chapters, and what each settles

CHAPTER	QUESTION	WHAT YOU LEAVE WITH
1 • Why Verification Exists	What does an escape cost?	Escape, respin, first-silicon success; the measured industry numbers; the cost curve that funds the whole discipline
2 • The Verifier's Mindset	Why can't the designer just test it?	Reconvergence, breakage thinking, independence of judgment, disciplined doubt
3 • The Verification Problem	Why is completeness impossible?	State-space arithmetic, controllability and observability, the oracle problem — and the pivot to risk
4 • The Verification Lifecycle	So how is it actually run?	Six phases, maturity stages, the review calendar, and what industry schedules really do

THE THREAD

Each chapter removes one comfortable belief — that correctness is the default, that testing your own work suffices, that "done" is reachable — and replaces it with something you can measure and defend.

01

Why Verification Exists

The problem, the numbers, the cost — and the design we will keep breaking for the rest of the book.

Eleven days before tape-out, one seed fails

A 2D transfer, **negative stride**, row crossing a **4 KB boundary**, mis-split by the DMA's legalizer — only while the **second channel** competes for the bus. 214 directed tests had passed for months.

Universe A — the regression caught it

- Overnight scoreboard mismatch
- Every internal signal observable
- **The fix takes a day**

Universe B — it shipped

- Silicon boots; bring-up sees nothing for weeks
- A customer corrupts a buffer, intermittently, under load
- Weeks of senior time, at a fraction of simulation's visibility
- **Respin, requalification, missed launch window**

THE WHOLE BOOK IN ONE LINE

The difference between these two universes is one nightly regression. Multiplied across every block of every chip, that difference is why verification consumes most of a project.

SAY Ask the room which universe their last project was in — and how they know.

Three words that price the discipline

TERM	MEANING
Bug escape	A design flaw that survives verification and reaches the next stage — integration, silicon, or the field
Respin	The consequence when an escape reaches silicon and cannot be worked around: a new fabrication cycle with corrected masks
First-silicon success	The fraction of projects whose first fabricated silicon is production-worthy — the effectiveness headline of the whole discipline

VERIFICATION VS VALIDATION VS TEST

Verification asks: *is the design right?* **Manufacturing test** asks: *was this copy built right?* **Validation** checks the actual silicon after fabrication.

THE NUMBER

14% of IC/ASIC projects achieved first-silicon success in **2024** — the lowest in twenty years. Six out of seven projects pay for at least one extra fabrication cycle.

Source: Wilson Research Group 2024 IC/ASIC functional verification trend report (597 eligible participants, ±4% at 95% confidence).

What the industry measures about itself

MEASURE	2014	2024	READING
First-silicon success (IC/ASIC)	~30%	14%	Decline unbroken from 2012; twenty-year low
Projects behind original schedule	61%	75%	Verification sits at the end of the critical path
Designs with an embedded processor	71%	84%	52% back in 2004 — each layer multiplies the intersections

What sends projects back to the fab

- **Logic or functional flaws** lead the categories — ahead of analog, timing, power, physical
- Root causes: **design errors** first, then **changing, incorrect or incomplete specifications**
- A large share of "hardware bugs" are ambiguities about what the design was *supposed* to do

What 2024 projects must also carry

- **95%** — two or more asynchronous clock domains
- **83%** — security features · **44%** — a safety-critical standard
- **58%** RISC-V · **59%** an AI accelerator

SAY Read "14%" as a headwind, not an indictment: complexity compounded while respin rates only slowly worsened. Projects using formal and mature methodologies reported fewer spins in 2024.

Sources: Wilson Research Group 2024 IC/ASIC report; Foster, "Trends in Functional Verification," DAC 2015 (2014 study, 1,886 participants).

FPGAs: the illusion of a free respin

No mask set, so a fix is "just" a rebuild. Asked in 2024 how many non-trivial bugs escaped into production, only **13%** of FPGA projects answered zero.

2024

87% of FPGA projects shipped production systems with non-trivial bugs — notably worse than the IC/ASIC first-silicon rate.

Why the comfort is false

- **Cultural diagnosis:** teams minimise pre-lab verification and race to the bench — a strategy that worked for simple parts, not for SoC-class devices
- **Reprogramming is not free:** in aerospace it means removing covers and a full system revalidation
- **The transition is under way:** FPGA verification headcount grew 8.5% CAGR against 4.7% for design, 2012–2024

THE RULE THAT FOLLOWS

Spend pre-lab effort **where the design is new**. Reuse of verified IP is what lets some projects legitimately spend less.

Source: Wilson Research Group 2024 FPGA functional verification trend report.

"70% of the effort" — folklore, and what was measured

The folklore

- Quoted in nearly every talk and textbook
- Also called "*subjective or unsubstantiated*" — repeated publication to publication without primary data
- Yet well rooted: ~70% of design effort in the SoC era;
~70% of design *time* on multiple projects; validation
~70% of SoC time and resources

THE DEFENSIBLE CLAIM

More than half — and how much more is set by how much of your design is new.

The measurement

57% of total project time spent in verification, measured in **2014**, essentially unchanged from 2012.

- Wide dispersion in *both* directions — and a growing tail above 80%
- Low end: projects assembling **pre-verified IP**
- High end: projects with high proportions of **newly developed IP**

The dispersion is the insight. The 70% figure is not wrong so much as unqualified.

SAY This is a lesson in reading industry numbers: a figure everyone repeats, a figure someone measured, and the honest sentence that survives both.

Whatever the org chart says, you are running a verification project

FINDING	WINDOW	NUMBER
Headcount growth, IC/ASIC	2007–2014	12.5% CAGR for verification engineers vs 3.7% for designers
Headcount growth, cooled	2016–2024	3% verification vs 2.4% design — from an already-inverted baseline
Peak team size	by 2014	Mean peak verification engineers exceeded mean peak design engineers
Processor segment	2024	A 5-to-1 verification-to-design ratio is <i>not unusual</i> (not a norm)
Designers verify too	2014 → 2024	47% → 49% of a design engineer's own time goes to verification

AND THE LARGEST SINGLE TIME SINK INSIDE VERIFICATION IS

Debug — ahead of planning, testbench development and running tests, in 2014 and again in 2024. It is also **unpredictable and varies wildly between projects**, which is why schedules extrapolated from the last project keep failing.

SAY Much of this book — self-checking testbenches, assertions at the point of origin, disciplined triage — is at bottom debug-time reduction.

The cost curve is an observability curve

Cost to find and fix **rises**; time available **shrinks**; and behind both sits how much internal state you can see when something goes wrong.

Block RTL simulation — any internal signal, any time · fix = edit and recompile

Full-chip simulation / emulation — reduced · fix = rebuild, hours

Post-silicon bring-up — scan and trace only · workaround or respin

Production / field — near zero · recall, errata, respin

- **Simulation is slow but transparent:** roughly a billion times slower than the target silicon — booting an operating system would take years
- **Post-silicon debug is measured in weeks:** on one heavily instrumented program, medium-severity bugs took ~**10 days** to root-cause, 90th percentile **20 days** — per bug
- **The industry pre-pays in silicon:** in some modern SoCs over **20% of die area** is design-for-debug instrumentation
- **The market window dwarfs everything:** missing a launch window is documented in the range of **millions to billions**
- The "10× per stage" rule is a *mnemonic, not data* — but no mechanism pushes the other way

Anatomy of a near-escape

Bug ID: SOC-1284

Title: DMA ch0 2D negative-stride transfer corrupts data at 4 KB boundary when ch1 is concurrently active

Found by: Nightly constrained-random regression (scoreboard mismatch), 11 days before tape-out

Severity: Critical - silent data corruption, no error response raised

Root cause: Midend transfer legalizer mis-splits a negative-stride 2D row that crosses a 4 KB AXI boundary; the malformed burst is masked unless ch1 owns the backend in the same window.

Escape analysis: 214 directed DMA tests passed; none combined negative stride x 4 KB crossing x dual-channel activity. The functional coverage model had no cross of these three dimensions - the hole was invisible to our own metrics.

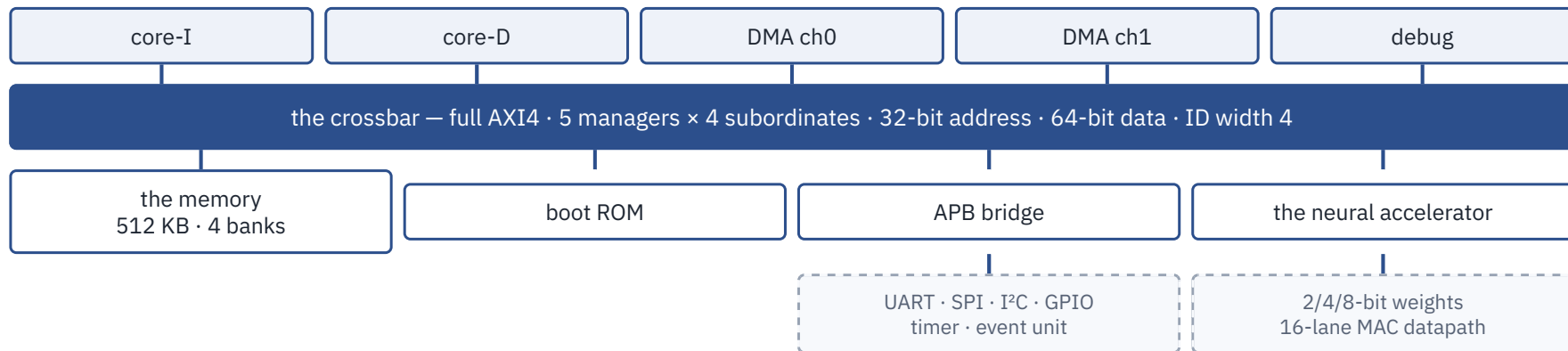
Cost if escaped: intermittent field corruption -> weeks of bring-up debug at ~10 days/bug post-silicon norms, likely respin + requal

Actions: (1) fix legalizer split logic; (2) add coverage cross stride_sign x boundary_cross x channel_concurrency; (3) extend scoreboard with end-to-end check for 2D descriptors; (4) audit all legalizer corner cases against the AXI 4 KB rule.

NOTE WHAT FOUND IT AND WHAT DID NOT

The directed tests all passed, honestly. Constrained-random found the unimagined combination; the **scoreboard**, not any human, noticed the corruption; and the **coverage model could not even represent the hole**.

The reference SoC — small enough to hold in your head



the core — 4-stage, in-order, single-issue RV32IMC, ~50k gates, optional DSP/SIMD extensions

the DMA engine — frontend / midend / backend, 2 channels, 1D and 2D, max burst 256 beats

clocking — 3 asynchronous domains: system 200 MHz, peripheral 50 MHz, always-on RTC 32 kHz

power — 2 UPF domains, retention on the event unit

Representative by the numbers: an embedded processor (like 84% of 2024 projects), RISC-V (58%), an AI accelerator (59%), multiple async clock domains (95%)

SAY Every element is a teaching seam: the crossbar carries protocol and formal, the DMA carries data-mover corner cases, the core carries ISA co-simulation, the clock and power domains carry CDC and low-power.

Questions for the room — the economics

1. Take the last bug that **escaped** on a project you worked on. At which stage was it found — and what did the lost observability cost, in days?
2. What fraction of engineering hours does your project spend on verification? Is that number **measured, or assumed**? What would it take to measure it?
3. If your first silicon works, was that **verification or luck**? Name one piece of evidence that would tell the two apart.

FACILITATION *Question 3 is the one that lands. Most teams discover they have no evidence, only an outcome — which is exactly the gap Chapters 5 to 7 close.*

02

The Verifier's Mindset

From "make it work" to "prove it does not break" — a different intellectual act, not a second pass over the same one.

Two minds, one specification

The DMA designer verifies her own block in one week — register defaults, 1D and 2D transfers, a bad address, completion interrupts. All pass. Three months later a random seed finds the negative-stride bug her tests could never have reached.

THE DESIGNER

Constructs a component that correctly implements the intended function, with the required performance and, ideally, without defects.

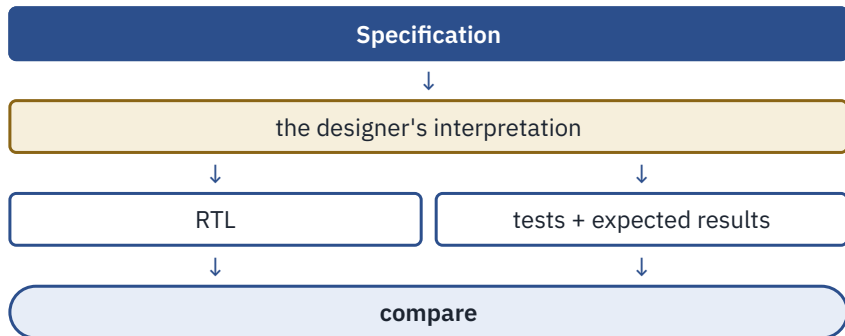
THE VERIFIER

Starts from the opposite premise — **the bugs are already in there** — and finds them by stressing protocols, exploring corner cases, and applying zero tolerance toward inconsistencies.

- Two **orientations toward the same specification** — not two intensities of one activity.
- Not a distinction between job titles: designers already spend **49%** of their time verifying (2024).
- Hiring filters on languages and tools. **Mindset** is the stronger determinant — and takes years, not months.

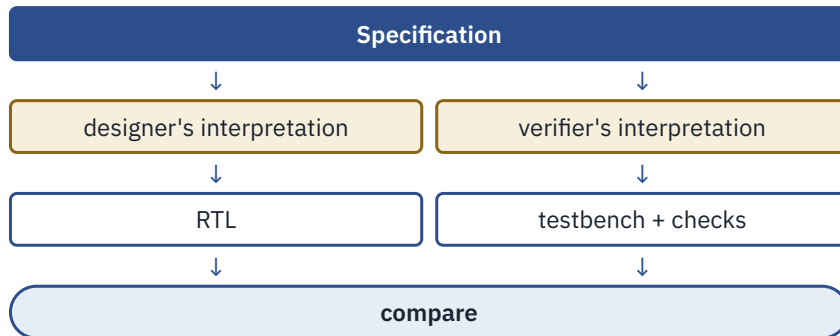
Reconvergence: where do the two paths meet?

Self-verification — they meet at the interpretation



If the interpretation is wrong anywhere, verification faithfully confirms that the design matches what she believed the spec to mean.

Independent verification — they meet at the spec



Three mechanisms reduce human error: automation, mistake-proofing, and **redundancy**. Design is too creative for the first two.

SAY Independence attaches to the artifact, not the person: a designer is often an excellent verifier of someone else's block. What redundancy forbids is only one mind on both paths.

Stress it — do not interoperate with it

THE PARADOX OF ROBUSTNESS

Build a monitor for a protocol needing clock-data recovery. **The more robust its recovery algorithm, the less checking it performs** — a tolerant monitor gracefully adapts to data rates that are out of spec, masking the bug it exists to catch. The right answer is the *least tolerant* observer the reference clock permits, even though no real receiver would ever be built that way.

Verification's aim is not to replicate reality but to verify the design in the most thorough manner possible — *"even if it means doing so in spite of reality."*

THE HELPFUL TESTBENCH	SYMPTOM	CURE
Handshakes	Auto-NACKs a corrupted transfer, so a simulation that should have failed keeps running	Log an error and fail; NACK generation only under explicit error-injection control
Status outputs	Tests pace themselves on the FIFO's <code>full</code> / <code>empty</code> flags — the blind leading the blind	Check the flags first (white-box assertion or independent count), <i>then</i> rely on them
Error recovery	Recovery paths keep a failing simulation alive; downstream checking is compromised anyway	Stop on first error — a mousetrap that has caught its bug is done for that simulation

"Invalid" is not a stimulus filter

Whether a scenario is valid or invalid is irrelevant. The only question is whether it **can happen** — and if it can, the design's response to it must be verified.

REPORTED CASE 1

A designer dismissed a **zero-radius circle** as meaningless input. In that very production system, upstream software was generating zero-radius circles routinely — unknown to him.

REPORTED CASE 2

A low-power mode bit was tested the obvious way: write 1, check entry; write 0, check exit. It passed until tape-out. Writing **0 to a bit that was already 0** put the device into low-power mode.

On the DMA

- "Negative stride" is on the list not because anyone plans to use it — but because **the register field is signed**, so it can happen
- The verifier crosses the breaking points: stride sign × boundary proximity × channel concurrency × burst length
- That cross is exactly the cell where the legalizer's bug lives

THE REFLEX TO UNLEARN

Nothing in the specification suggests either test above. Both bugs are invisible to a mind that tests what the design is *for*, and near-inevitable finds for a mind that enumerates what the design can be *subjected to*.

Attacking the transfer legalizer

The designer thinks *"the legalizer splits at 4 KB."* The verifier states the **prohibition at the output** and lets the tool hunt for any way to violate it.

```
// No INCR write burst emitted by the DMA backend may cross a 4 KB page,
// regardless of how the midend legalized the transfer. (FIXED bursts hold
// their address; WRAP bursts stay inside their aligned container.)
// 16-bit arithmetic on purpose: an illegal awsize (>3 on this 64-bit bus)
// must overflow the comparison and fail - not wrap silently and pass.
property p_no_4kb_crossing;
  @(posedge clk) disable iff (!rst_n)
    (awvalid && awready && (awburst == 2'b01)) |->
      (16'(awaddr[11:0]) + ((16'(awlen) + 16'd1) << awsize)) <= 16'd4096;
endproperty
a_no_4kb_crossing : assert property (p_no_4kb_crossing);
```

THE PROPERTY

The zero-tolerance check: one rule, every cycle, every test.

THE CROSS

The map of where to apply pressure: **stride sign × distance to boundary × burst length × other channel active** — a block of empty bins would have exposed the directed suite's blind spot long before any failure.

SAY *The 16-bit arithmetic is doubt applied to the verifier's own code — so an illegal size fails loudly instead of wrapping past the check and passing.*

Independence of judgment, and the value of doubt

Three disciplines that protect independence

- **Predict from the specification, not from the design.** The moment your checker is tuned until the design passes, you are verifying the design against itself.
- **Trust no signal you have not checked.** Every unchecked DUT output delegates a piece of your judgment back to the object under suspicion.
- **Speak as the third party in the room.** The verifier is the natural liaison between architect and designer — often the first to detect that the two have diverged.

"SOFTWARE WILL NEVER DO THAT"

The operative question is not intent but reachability: **can the programming interface express it?** If yes, either the RTL is fixed or the case is detected and rejected — and a test proves the rejection.

Four doubts

- **Doubt the green.** No coverage without checking. A bin hit while nothing verified the outcome is worse than an empty bin — it converts ignorance into confidence.
- **Doubt the quiet.** A block that has never failed a test is either simple or under-attacked. Ask which, and reallocate.
- **Doubt your own tools.** Much of a testbench's behaviour never appears in waveforms; if you cannot interrogate it from the logs, you have no grounds to trust its verdicts.
- **Doubt the schedule's promises.** Success undefined is unfalsifiable. *Doubt, written down and agreed before the pressure arrives, is called a plan.*

Questions for the room — the mindset

1. Pick a block you have verified. **Where did your two paths reconverge** — at the specification, or at somebody's interpretation of it?
2. Name one **DUT output your environment trusts without checking**. What would a bug in that output do to your pass rate?
3. Which of your blocks has been **quiet longest**? Is it simple, or under-attacked — and what evidence do you have either way?

FACILITATION *Question 2 usually produces a ready flag or a status register within thirty seconds. That is the exercise working.*

03

The Verification Problem

Why completeness is not merely expensive but impossible — and what the discipline does about it.

"Test all the cases." Let us count them.

One channel of the reference SoC's DMA engine — the block a manager might reasonably call simple:

REGISTER	BITS	ROLE
SRC_ADDR · DST_ADDR	32 + 32	base addresses
NUM_BYTES	32	bytes per row
SRC_STRIDE · DST_STRIDE	32 + 32	signed 2D strides
NUM_REPS	16	2D row count
CFG · STATUS	8 + 8	flags, error code

192 bits of architecturally visible state → $2^{192} \approx 6.3 \times 10^{57}$ configurations, for one channel.

NOW PUT A NUMBER ON "VISITING" THEM

1 million new configurations per second × a farm of 1,000 simulations = $10^9/\text{s}$.

The universe is $\sim 4.4 \times 10^{17}$ seconds old.

Running since the Big Bang, that farm reaches 4.4×10^{26} configurations — a fraction of the space around 10^{-31} .
Not one percent. Not one billionth. A denominator of thirty-two digits.

And the register file is the *shallow* part

- The two address registers alone: $2^{64} \approx 1.8 \times 10^{19}$ pairs — nearly **six centuries** on that farm
- One 16 × 64-bit backend FIFO: 1,024 bits → $2^{1024} \approx 10^{308}$
- Two channels: $2^{384} \approx 3.9 \times 10^{115}$ — where the canonical DMA bug lives
- Sequences too: ten binary environment choices per cycle, a thousand cycles → one path out of 2^{10000}

SAY Not a matter of budget: enumeration is off the table before the project starts — and 512 KB of SRAM alone holds over four million state bits.

Abstraction shrinks the space — and it is still too big

Most states are equivalent. So partition each dimension into **equivalence classes** and cross classes instead of bits — for one DMA channel:

DIMENSION	CLASSES
source offset in 4 KB page	~16
destination offset in page	~16
length (sub-beat ... crosses 2+ pages)	~10
stride (sign × relation to row, zero)	~6
repetitions (1, 2, many, max)	~4
other channel (idle / active / erroring)	~4
backend back-pressure (ready delay 0–7)	~8
error response (none / read / write / both)	~4

The product is about **eight million cases** — no longer cosmological, still far beyond directed testing. A team maintaining a thousand hand-crafted tests is exceptional.

THE ACTUAL STRUCTURE OF SIMULATION-BASED VERIFICATION

1. Abstraction 2. Stimulus automation 3. Measurement.

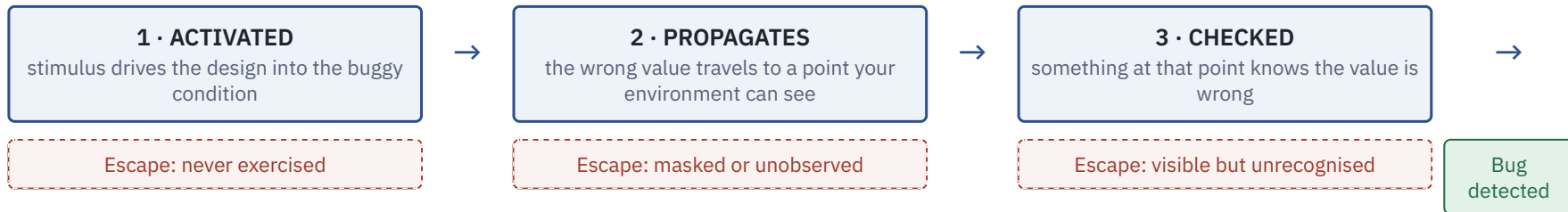
Name the bargain exactly: relative completeness comes from a coverage metric, and defining that metric is *"a selective, and thus subjective, process."*

Doesn't formal verification solve this?

- **Partly, and importantly:** a proven property holds for *every* reachable state and input sequence.
- But model checking is exhaustive state-space search, worst-case **exponential in storage elements** — the same wall in formal clothes.
- So full proofs go to small or well-abstracted targets: arbitration, protocols, control kernels.
- **Bounded proofs** are honest: guaranteed only within the bound.
- Rigorous *complete verification* works — purchasable for selected modules, never ambient.

A bug is not found when it exists

It is found when three things happen in sequence. Break any link and the bug survives the test that technically "executed" it.



LINK 3 BROKEN – VISIBLE, NEVER CHECKED

The UART's RX overrun flag is never cleared if software reads status in the same cycle a stop-bit error arrives. For months the condition was *activated* in regressions. No checker ever looked at the flag after that sequence.

LINK 2 BROKEN – NOTHING TO PROPAGATE THROUGH

The event unit's retention register loses its configuration on power-domain re-entry when a wake event races isolation release. Plain RTL simulation did not model the power behaviour at all — it escaped RTL entirely.

Controllability and observability

CONTROLLABILITY

Your ability to **steer** the design into a given internal condition, using only the interfaces you can drive.

OBSERVABILITY

Your ability to **see** the consequences of an internal event, at the interfaces you can watch.

Smaller targets offer more of both. Verify each feature at a level where you can provoke it *and* see it fail.

Both collapse on the way to silicon

PLATFORM	WHAT YOU CAN OBSERVE
RTL simulation	Virtually any internal signal, at any time
FPGA prototype	A few thousand signals, pre-selected before the bitstream is built
Silicon	Order of a hundred signals routed to trace buffers and pins; control limited to hooks defined in advance

Change the design, don't be a hero

- Observability problem → sample points readable through memory-mapped registers
- Controllability problem → provisions to **force error and exception conditions**
- Assertions do it from inside: the error is flagged at its birthplace

The oracle problem: how do you know the response is right?

A simulator produces what the design *did*. Correctness relates that to what it *should have done* — and "should" lives in prose, in the architect's head, or nowhere. **Every practical oracle is partial.**

ORACLE	WHAT IT CHECKS	WHAT IT IS STRUCTURALLY BLIND TO
Reference model	An independent executable of the specification, compared continuously — golden by definition for processors	Anything outside its projection: performance, protocol legality, timing side channels
Scoreboard	Data integrity and completeness of a transform: nothing lost, nothing invented	<i>When</i> things happened (latency, fairness, starvation) and <i>how</i> (a protocol violation delivering correct data)
Assertions	Local invariants and protocol obligations, every cycle, on every test — the sharpest partial oracle	Everything outside the one rule each assertion states
Offline / human	Post-simulation comparison; some responses are easy for a human, awkward for a program	Arrives far from the state that caused it — diagnosis is cheap only near the failure

THE FAILURE THAT DEFEATS ALL ORACLES AT ONCE

The common-mode error. If the scoreboard author misreads the same ambiguous sentence the designer misread, model and design agree and the bug is structurally invisible. A passing regression says the design and the testbench agree — not that either is right.

What one assertion proves — and what it does not

Interrogate the 4 KB property from Chapter 2, deliberately, question by question.

QUESTION

DOES A_NO_4KB_CROSSING ANSWER IT?

Did any issued burst cross a 4 KB boundary?

YES Every INCR burst, every cycle, every test — FIXED and WRAP cannot cross by construction

Did the split bursts carry the right data?

NO It never looks at data

Do the split bursts add up to the programmed transfer?

NO It sees bursts one at a time, not their sum

Does a legal transfer ever complete at all?

NO A backend that hangs forever satisfies it *vacuously*

Was the *read* channel also legal?

NO Separate assertion needed

THE DIVISION OF LABOUR, ON THE CANONICAL BUG

The emitted bursts were individually **legal** — the assertion stayed silent. The **data** landed wrong, and only the scoreboard's byte-accurate memory model caught it. Assertion and scoreboard each covered the other's blind spot.

SAY The question to internalise is not "which oracle is best" but "which projection of correctness is each oracle responsible for, and is any projection unowned?"

From exhaustive to risk-driven

Assemble the three results

- You cannot **visit** all the states — finite, yet unvisitable
- You cannot **reach or see** everything even in the fragment you do visit
- You cannot fully **check** even what you see — every oracle is a partial projection of an imperfect specification

ONLY ONE CONCLUSION IS AVAILABLE

Verification can show the presence of errors, not their absence. Given enough time, an error *will* be found. So the question becomes:

"Is the error likely to be severe enough to warrant the effort spent identifying it?"

What changes the morning after

- **Effort goes where risk is, not where habit is.** Risk = likelihood × escape cost. Likelihood concentrates in novelty and intersections; cost concentrates where escapes are expensive to find downstream.
- **Techniques are matched to the shape of each risk.** Random simulation covers the reachable majority fast; directed tests reach a known corner precisely at engineer-days each; formal is exhaustive per property where it converges.
- **"Done" is redefined.** Not "verified everything" but *"reached the agreed evidence threshold on every planned item, and the risks we chose not to retire are enumerated and accepted by people entitled to accept them."*

You are never finished. You stop, on purpose, in the open.

SAY *Teams that pretend otherwise still stop — the tape-out date sees to that — but they stop with their residual risk unexamined.*

Questions for the room — the problem

1. Compute the **state space of a block you own**, the way we did for one DMA channel. What does that number change about your test plan — concretely?
2. Take one feature of that block. **Which oracle owns it**, and name one thing that oracle is blind to. Who owns *that*?
3. Name a risk your last project **accepted**. Was it written down? Who signed it? If nobody, the schedule signed it for you.

FACILITATION *Question 1 is a five-minute exercise with a pen. The number is always larger than the room expects, and the silence afterwards does the teaching.*

04

The Verification Lifecycle

From the plan to the sign-off: phases, maturity stages, reviews — and what schedules actually do.

Six phases, three loops — once per verification level



LOOP 1

A bug in the middle reveals a feature missing from the plan → **back to planning.**

LOOP 2

Coverage holes at closure send you **back to regression.**

LOOP 3

An escape after sign-off must flow into the **next project's plan**, or you pay for the same lesson twice.

ONE LIFECYCLE PER LEVEL

The crossbar runs its own complete cycle at block level; the SoC-level cycle *consumes* its results and gates its own bring-up on the crossbar reaching maturity. Levels exist to put each class of bug where it is cheapest to find.

Milestones, not percentages

UNCHECKABLE

"Agent coding 80% done."
"We're running lots of tests."
"Soon."

EITHER TRUE OR FALSE

"A single packet goes through the DUT and the scoreboard checks it."
"The register reset-value test passes in regression."

Lifecycle milestones: plan review · first transaction · feature complete · RTL freeze · coverage closure · verification sign-off. The vocabulary varies; the pattern does not.

Planning is the highest-leverage phase

- Verification effort runs **100% to 200% of the RTL design effort**. No team would spend the design half without a specification — the verification plan *is* that specification.
- **Audit your reuse:** for every piece of collateral, is it *reused* / *modified* / *new*? Assuming "reuse" means 100% reuse is how projects arrive at bring-up under-scoped.
- **Plan in deliverables with a definition of done** that is an action, not a state.
- **Reported ROI** (one organization's figures): 10–20% added to the verification front end, returning 10–40% of the overall schedule and at least 20% of implementation and closure effort.

SAY Treat the ROI numbers as one team's experience. The direction — planning is the highest-return phase — is consistent across every source in the chapter.

The maturity ladder — and a stage exit in the open

Stage 3 — Verified / sign-off ready

Coverage closed against the plan, every exclusion reviewed; regression passing over a sustained window; every open bug dispositioned; the sign-off review has accepted the evidence.

Stage 2 — Verification mature

All planned stimulus and checkers implemented; coverage model reporting; regression at cadence with a managed pass rate; bug-open curve past peak. *The gate others build on.*

Stage 1 — Bring-up complete

Plan reviewed; the environment drives real transactions against current RTL and checks them automatically; a smoke suite passes in regression.

Each line is **objectively true or false**, so a stage claim is auditable in an hour by an outsider — and the stages compose upward into an SoC dashboard.

The crossbar's stage-2 exit checklist, week 16

- [x] Verification plan reviewed; actions closed
- [x] All 5x4 manager/subordinate paths driven and checked
- [x] Protocol assertions bound on all 9 ports; sim + formal
- [x] Coverage model XB-01..XB-07; reporting nightly
- [x] Nightly regression ≥ 300 seeds, pass $\geq 95\%$
- [x] Bug-open curve past peak (closure $>$ discovery)
- [] Formal ordering proofs (XB-04): 2 still bounded-only
- [] Starvation metric (XB-05) not yet on dashboard

Two open boxes: the crossbar has **not exited** stage 2 — and everyone sees exactly why.

THE WARNING

Stages describe **verification maturity, not RTL quality**. A block can be stage 2 with fifty open bugs: the claim is that the machinery to find them is mature.

SAY A stage claimed without its checklist is a percentage wearing a costume.

Who checks what, and when

Reviews are cheap, they are skipped first under pressure, and they catch the class of error no tool can — the error of **intent**.

REVIEW	WHEN	WHO	WHAT IS BEING JUDGED
Requirements / plan	Architectural spec complete	Design, verification, software, systems	Is the feature list complete? Are the must-not-do's and out-of-scopes right?
Assumptions	During planning; re-run when the spec moves	Verification with the designers, face to face	Every unstated "someone else covers this", made explicit and agreed
Environment / code	Continuously through build	Verification peers	Is the testbench itself trustworthy? It is a design too
Coverage model	Before the long middle	Designers + verification	Does the model measure the <i>intent</i> , not the implementation?
Progress / triage	Weekly through the middle	Lead + team	Trend lines: bug curves, coverage, regression health
Sign-off	End of closure, gating tape-out	All of the above + management	The full evidence package against the plan; explicit acceptance of residual risk

WHY THE OUTSIDER MATTERS

At the crossbar's plan review the firmware engineer — invited under protest — asked how the debug port is verified while the core is halted. Silence. One row was added; in week 16 the test it produced caught an arbiter assumption that would have made post-silicon debug intermittently unreliable.

The crunch, the guillotine, and the gap you created

What effort actually does

- Planning and build spend at a modest, steady rate; the long middle burns compute and debug time
- **Closure concentrates the highest-pressure work** — last coverage holes, intermittent failures, hardest bugs — into the weeks when the date is least movable. That is the **crunch**.

THE GUILLOTINE

When a fixed ship date simply cuts off whatever verification was still in flight, what ships is *"whatever happened to work"* — and the features in the basket may be the ones the market needed.

The defence was built in week one: a prioritised plan converts the guillotine into a negotiation, and a signature on the risk register converts "we ran out of time" into an accountable decision.

Recall the ambient numbers: 75% of projects behind schedule and 14% first-silicon success (2024); 87% of FPGA projects shipping non-trivial escapes (2024).

The verification gap, decomposed

CLASS	WHAT TO DO ABOUT IT
Inherent	Complexity you must absorb — <i>innovate</i>
Transient	Skills and tooling that lag but catch up — <i>invest</i>
Self-induced	Schedule pressure forcing shortcuts; best-known methods unused (estimated ~20% productivity cost); sloppy environment engineering — <i>introspect</i>

Individual productivity spread across engineers is reported up to **10x**, which makes team composition a first-order schedule variable no methodology hides.

Much of this book aims at the third class: the gap you created and can close without waiting for anyone's roadmap.

What Part I established

- **Verification is economics.** Bug cost rises and fix time shrinks at every stage.
- **Mindset, not tools, determines effectiveness.** Verifying your own design reconverges on your interpretation, not the specification.
- **Completeness is not on offer.** The space is unvisitable, observability is bounded, every oracle is partial.
- **So "done" is a risk-acceptance decision,** made in the open — which is what the lifecycle exists to produce.

THE PIVOT TO CARRY FORWARD

Replace "*have we verified everything?*" (always no) with "*is the residual risk known, bounded, and accepted?*"

Read the chapters

- | | |
|---|---|
| 1 | Why Verification Exists — the numbers, the cost curve, the reference SoC |
| 2 | The Verifier's Mindset — reconvergence, breakage, independence, doubt |
| 3 | The Verification Problem — state space, observability, oracles, the pivot |
| 4 | The Verification Lifecycle — phases, maturity stages, reviews, schedule |

NEXT

Part II — Planning and Measurement. The plan as a contract, the measurement as truth. Without both, verification is opinion.